📖 DAY 5: COMPLETE TOPIC-BY-TOPIC STUDY GUIDE

Generative AI, Large Language Models (LLMs), Habana Gaudi & OpenVINO Acceleration

GLA University — Department of Computer Science & Engineering | BCSG701

📄 TABLE OF CONTENTS

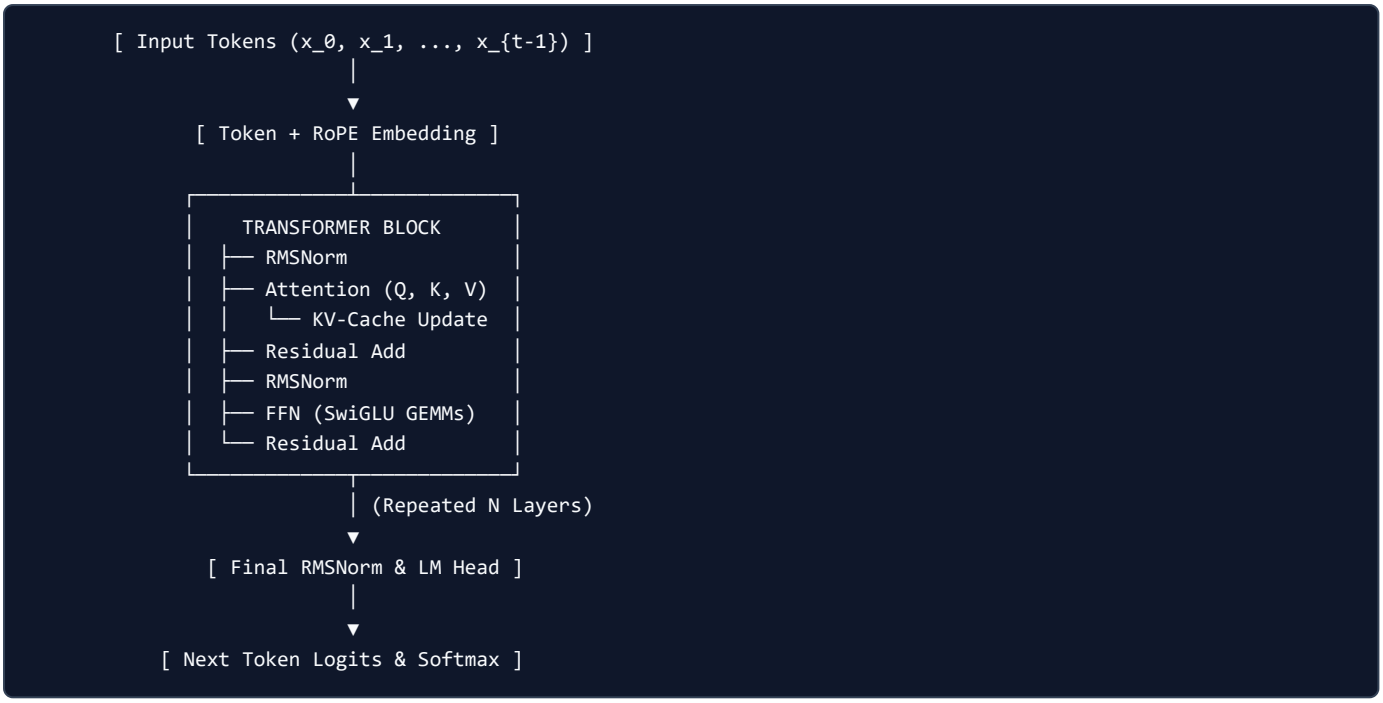
1. Module 1: Introduction to Generative AI & Foundation Models on HPC
2. Module 2: LLM Inference Physics: Prefill vs Decode & The Memory Wall
3. Module 3: Intel Xeon Scalable Processors & Advanced Matrix Extensions (AMX)
4. Module 4: Habana Gaudi 2 & Gaudi 3 AI Accelerators
5. Module 5: Intel OpenVINO Toolkit & Model Optimization Architecture
6. Module 6: Weight-Only Quantization (INT8 & INT4) with NNCF
7. Module 7: Parameter-Efficient Fine-Tuning (PEFT) & Low-Rank Adaptation (LoRA)
8. Module 8: Multi-Node Slurm Orchestration for Distributed LLM Training & Serving

MODULE 1: Introduction to Generative AI & Foundation Models on HPC

1.1 The Transformer Architecture in High-Performance Computing

Modern Large Language Models (LLMs)—including LLaMA, Qwen, Mistral, and GPT-based architectures—are built upon stacked decoder-only Transformer layers. Each Transformer layer consists of two dominant sub-layers:

1. **Multi-Head Self-Attention (MHA) / Grouped-Query Attention (GQA)**: Computes relationships between all pairs of tokens in the active context window.
2. **Feed-Forward Network (FFN / SwiGLU)**: Applies high-dimensional non-linear projections to enrich token representations.



1.2 Mathematical Foundations of Multi-Head Attention

Given an input sequence representation $\mathbf{X} \in \mathbb{R}^{B \times L \times d_{\text{model}}}$, the model projects $\mathbf{X}$ into Query, Key, and Value matrices using learned projection weights $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

For each attention head $h \in \{1, \dots, H\}$, scaled dot-product attention is evaluated:

$$\text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^T}{\sqrt{d_k}}\right)\mathbf{V}_h$$

Where:

- B is the batch size.
- L is the context length (sequence length).
- d_{model} is the hidden dimension (e.g. 4096 for a 7B model).
- $d_k = d_{\text{model}}/H$ is the per-head dimension.

1.3 The Quadratic Context Bottleneck

Computing $\mathbf{Q}\mathbf{K}^T$ requires an outer product resulting in an $L \times L$ attention score matrix per head.

- **Compute Complexity**: $O(B \cdot H \cdot L^2 \cdot d_k)$
- **Memory Complexity**: $O(B \cdot H \cdot L^2)$

At $L = 2,048$, $L^2 \approx 4.19 \times 10^6$. At $L = 32,768$, $L^2 \approx 1.07 \times 10^9$ (a $256\times$ explosion in memory and operations!). In modern supercomputing clusters, FlashAttention and PagedAttention techniques are implemented to compute attention in tiled blocks matching CPU/GPU on-chip cache boundaries, bypassing the allocation of full $L \times L$ attention matrices.

MODULE 2: LLM Inference Physics: Prefill vs Decode & The Memory Wall

2.1 The Two Phases of Auto-Regressive Generation

Inference on generative language models operates across two fundamentally distinct computational regimes:

PREFILL PHASE VS DECODE PHASE		
Operational Property	Prefill Phase (Prompt)	Decode Phase (Token Gen)
Input Sequence	Entire Prompt (L_{prompt} tokens)	Single Token (1 token)
Computational Style	Matrix-Matrix Multiplication	Vector-Matrix Product
Operational Regime	**Compute Bound**	**Memory Bandwidth Bound**
Arithmetic Intensity	High ($> 15\ \text{FLOP/byte}$)	Very Low ($\approx 0.5\ \text{FLOP/byte}$)
Hardware Limiter	Vector/Systolic TFLOPs	DRAM / HBM Bandwidth (GB/s)
Primary Metric	**Time to First Token (TTFT)**	**Tokens Per Second (TPS)**

2.2 The Memory Wall: Why Decode is Bandwidth-Bound

Consider generating a single token from a 7-billion parameter model in 16-bit precision (FP16 or BF16):

- **Model Weight Size**: $7 \times 10^9 \times 2 \text{ bytes} = 14 \text{ GB}$.
- **Compute Required**: $2 \times 7 \times 10^9 = 14 \text{ GFLOPs}$ (one multiply-accumulate per parameter).
- To generate just **one single token**, the entire 14 GB model must be loaded from external DRAM into the processor caches!

$$\text{Arithmetic Intensity}_{\text{Decode}} = \frac{14 \times 10^9 \text{ FLOPs}}{14 \times 10^9 \text{ Bytes}} = 1.0 \text{ FLOP/byte}$$

On an enterprise server with 300 GB/s memory bandwidth:

$$\text{Max Theoretical TPS} = \frac{\text{Memory Bandwidth (GB/s)}}{\text{Model Size (GB)}} = \frac{300 \text{ GB/s}}{14 \text{ GB}} \approx 21.4 \text{ Tokens/second}$$

Notice that CPU or GPU clock speed is completely absent from this equation! Even if compute capacity were infinite, text generation cannot exceed 21.4 tokens per second due to the physical DRAM bandwidth limit.

2.3 The KV-Cache Formula

During auto-regressive generation, to avoid recomputing $\mathbf{K}$ and $\mathbf{V}$ tensors for historical tokens on every new step, tensors are cached in RAM:

$$\text{KV-Cache (Bytes)} = 2 \times B \times L \times n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times \text{bytes_per_element}$$

For a 70B model ($n_{\text{layers}} = 80, n_{\text{kv_heads}} = 8, d_{\text{head}} = 128$, FP16) with batch size $B = 16$ at context length $L = 4096$:

$$\text{KV-Cache} = 2 \times 16 \times 4096 \times 80 \times 8 \times 128 \times 2 = 21,474,836,480 \text{ bytes} \approx 21.47 \text{ GB}$$

The KV-cache consumes more RAM than the model weights themselves when concurrency scales!

MODULE 3: Intel Xeon Scalable Processors & Advanced Matrix Extensions (AMX)

3.1 Intel AMX Microarchitecture

Introduced in 4th Generation Intel Xeon Scalable Processors (Sapphire Rapids) and expanded in 5th Gen (Emerald Rapids) and Xeon 6, **Intel AMX** is a revolutionary hardware accelerator integrated into each processor core designed specifically for tensor operations and deep learning GEMMs.

```
+-------------------------------------------------------------------------+
|                     INTEL XEON CORE MICROARCHITECTURE                   |
+-------------------------------------------------------------------------+
|  [ Fetch & Decode Pipeline ]                                            |
|        |                                                                |
|        ▼                                                                |
|  [ Out-of-Order Execution Engine ]                                      |
|        ├── ALUs & Branch Units                                          |
|        ├── AVX-512 Vector Execution Units (FMA 512-bit)                 |
|        |                                                                |
|        └──► [ INTEL AMX HARDWARE ACCELERATOR ]                          |
|              ├── 8 x 2D Tile Registers: TMM0 - TMM7 (1 KB each = 8 KB total)  |
|              └── Tile Matrix Multiply Unit (TMUL):                      |
|                  ├── 1024 INT8 Operations / cycle                       |
|                  └── 512 BF16 Operations / cycle                        |
+-------------------------------------------------------------------------+
```

3.2 AMX Tile Registers (`TMM0` – `TMM7`) & TMUL

- **Tile Architecture**: AMX provides 8 2D register files named `TMM0` through `TMM7`. Each tile holds a 2D matrix of up to 16 rows and 64 bytes per row (1,024 bytes per tile).
- **TMUL Unit**: Performs fused multiply-add operations on matrix tiles:

$$C = C + A \times B$$

  Where $A \in \mathbb{R}^{16 \times K}$, $B \in \mathbb{R}^{K \times 16}$, and $C \in \mathbb{R}^{16 \times 16}$.
- **Throughput**: TMUL executes up to $2,048$ integer operations or $1,024$ floating-point operations per cycle per core, delivering up to an $8\times$ **throughput leap** over standard AVX-512 FP32 operations.

3.3 Linux Kernel OS Support for AMX

Because AMX tile registers add 8 KB of architectural state per thread, Linux requires explicit permissions before saving/restoring AMX registers during OS context switches:

```
#include <sys/syscall.h>
#include <asm/prctl.h>
#define ARCH_REQ_XCOMP_PERM 0x1023
#define XFEATURE_XTILECFG   17
#define XFEATURE_XTILEDATA  18

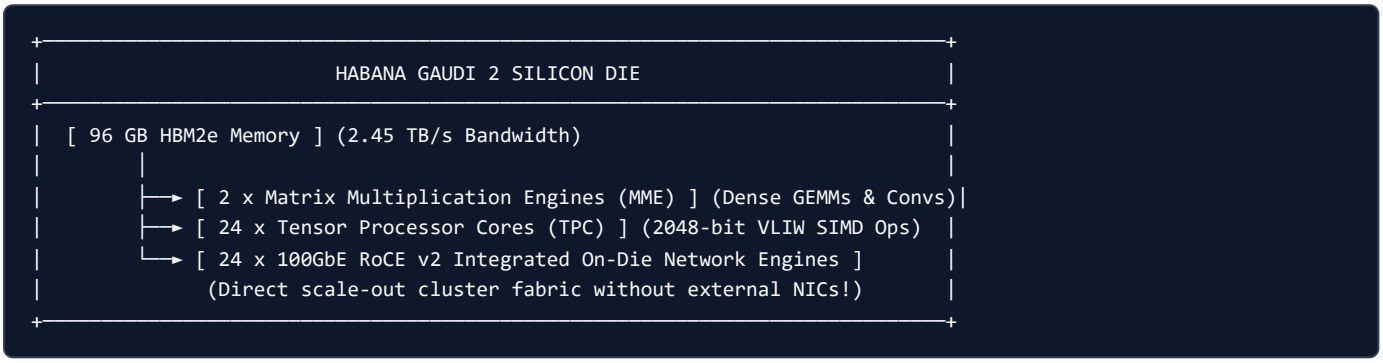
// Request dynamic AMX permission from Linux Kernel
syscall(SYS_arch_prctl, ARCH_REQ_XCOMP_PERM, XFEATURE_XTILEDATA);
```

MODULE 4: Habana Gaudi 2 & Gaudi 3 AI Accelerators

4.1 Habana Gaudi Architectural Philosophy

Designed from first principles for deep learning by Habana Labs (an Intel company), Gaudi departs from legacy GPU architectures by prioritizing two critical bottlenecks in distributed AI:

1. **Direct Tensor Operations**: Dedicated Matrix Multiplication Engines (MMEs) paired with fully programmable Tensor Processor Cores (TPCs).
2. **On-Die Cluster Interconnect**: Eliminating external PCIe network interface cards (NICs) by integrating 24 100GbE (Gaudi 2) or 200GbE (Gaudi 3) RDMA network engines directly on the processor die.



4.2 Gaudi 2 vs Gaudi 3 Technical Comparison

Feature	Habana Gaudi 2	Habana Gaudi 3
Process Node	TSMC 7nm	TSMC 5nm
TPC Cores	24 TPCs (VLIW SIMD)	64 TPCs
Matrix Engines (MME)	2 MME engines	8 MME engines
Memory Capacity	96 GB HBM2e	128 GB HBM3
Memory Bandwidth	2.45 TB/s	3.70 TB/s
FP8 Peak Compute	865 TFLOPs	1,835 TFLOPs
BF16 Peak Compute	432 TFLOPs	917 TFLOPs
Integrated Networking	24 x 100GbE RoCE v2	24 x 200GbE RoCE v2
Software Stack	SynapseAI + HCCL	SynapseAI + HCCL

4.3 On-Die 24x 100GbE / 200GbE RoCE Networking

In conventional GPU clusters, each server requires 8 discrete PCIe InfiniBand or Ethernet NICs ($8 \times \$1,500 = \$12,000$ per node in network adapters alone!) connected through motherboard PCIe switches. Gaudi integrates 24 RDMA network interfaces directly onto the silicon package:

- 21 ports are dedicated to all-to-all intradomain meshing across the 8 Gaudi cards in the server node.
- 3 ports connect directly to standard top-of-rack (ToR) Ethernet switches for multi-node supercomputing clusters.
- Eliminates expensive proprietary InfiniBand switches, slashing cluster capital expenditure (CapEx) by over 50%.

MODULE 5: Intel OpenVINO Toolkit & Model Optimization Architecture

5.1 OpenVINO Architecture Overview

The **Intel OpenVINO (Open Visual Inference and Neural Network Optimization) Toolkit** optimizes, compiles, and deploys deep learning models across heterogeneous Intel hardware:

1. **Frontend Converters**: Ingests models from PyTorch, ONNX, and TensorFlow.
2. **Intermediate Representation (IR)**: Normalizes models into a hardware-agnostic computation graph:
   - `model.xml`: XML topology representation (layers, shapes, connections).
   - `model.bin`: Contiguous binary file storing weights and biases.
3. **Execution Plugins**: Hardware-specific backends for CPU (Xeon AVX-512/AMX), GPU (Intel Arc / Flex / Max), and NPU (Intel Core Ultra).

```
[ PyTorch / HuggingFace Model ]
                │
                ▼ (ov::convert_model)
[ OpenVINO Intermediate Representation (.xml + .bin) ]
                │
                ├──► Graph Optimizer (Constant Folding, Dead Code Removal, Operator Fusion)
                ├──► NNCF Compression (Weight-Only INT8 / INT4 Quantization)
                ▼
[ Compiled Model (ov::Core::compile_model) ]
   ├── CPU Plugin (AVX-512 / AMX / oneDNN)
   ├── GPU Plugin (Intel Xe Matrix Extensions / oneDNN)
   └── NPU Plugin (Low-Power Inference Engine)
```

5.2 Key Compiler Graph Optimizations

- **Operator Fusion**: Fuses multiple sequential operations into a single execution kernel. For example, fusing Attention Scaling + Bias Addition + Softmax, or fusing Linear Projection + RoPE Rotation. This keeps intermediate tensors inside CPU L1/L2 caches, eliminating millions of DRAM read/write cycles.
- **Constant Folding**: Precomputes static subgraphs at compile time. Static tensor shape manipulations and weight transformations are executed once and saved into `model.bin`.

MODULE 6: Weight-Only Quantization (INT8 & INT4) with NNCF

6.1 Understanding Weight-Only Quantization

In deep learning training, 32-bit floating-point (FP32) is standard. In LLM inference, moving 14 GB of FP16 parameters to generate a single token chokes memory buses. **Weight-Only Quantization (WOQ)** converts the model weights from 16-bit floating point down to 4-bit or 8-bit integers while keeping intermediate activations in 16-bit precision:

- **Storage & Bandwidth**: 4-bit weights reduce model size by $4\times$ (a 14 GB model becomes 3.5 GB!).
- **Dynamic Dequantization**: During execution, the 4-bit weights are read from DRAM ($4\times$ faster), dequantized to FP16 inside CPU cache, and multiplied against activations.

6.2 Mathematical Mapping: Symmetric vs Asymmetric Quantization

- **Asymmetric Quantization**:

$$x_q = \text{clamp}\left(\text{round}\left(\frac{x}{S}\right) + Z, q_{\min}, q_{\max}\right)$$

Where S is the floating-point scale factor and Z is the integer zero-point offset:

$$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad Z = \text{round}\left(\frac{-x_{\min}}{S}\right) + q_{\min}$$

- **Symmetric Quantization**: Enforces $Z = 0$, centering the dynamic range at zero ($S = \frac{\max(|x_{\min}|, |x_{\max}|)}{q_{\max}}$). Symmetric is faster on hardware because no zero-point subtraction is required during multiply-accumulate.

6.3 Quantization with Intel NNCF

Using OpenVINO's Neural Network Compression Framework (NNCF):

```
import nncf
import openvino as ov

core = ov.Core()
model = core.read_model("qwen_openvino_ir/model.xml")

# Apply Asymmetric INT4 Weight-Only Compression
compressed_model = nncf.compress_weights(
    model,
    mode=nncf.CompressWeightsMode.INT4_ASYM,
    group_size=128,          # Quantize in blocks of 128 weights to preserve accuracy
    ratio=0.8                # Quantize 80% of layers to INT4, retain sensitive layers in INT8
)
ov.save_model(compressed_model, "qwen_openvino_int4/model.xml")
```

MODULE 7: Parameter-Efficient Fine-Tuning (PEFT) & Low-Rank Adaptation (LoRA)

7.1 The Cost of Full-Parameter Fine-Tuning

Fine-tuning every weight in an LLM requires storing optimizer states for every parameter. Using the standard Adam optimizer in FP32:

- Model Weights (FP16): 2 bytes/param
- Gradients (FP16): 2 bytes/param
- Adam First Moment m (FP32): 4 bytes/param
- Adam Second Moment v (FP32): 4 bytes/param
- Master Weights (FP32): 4 bytes/param
- **Total per Parameter**: 16 bytes!

For a 70B parameter model: $70 \times 10^9 \times 16 \text{ bytes} = 1.12 \text{ Terabytes}$ of memory! Full fine-tuning is impossible without massive multi-node supercomputing clusters.

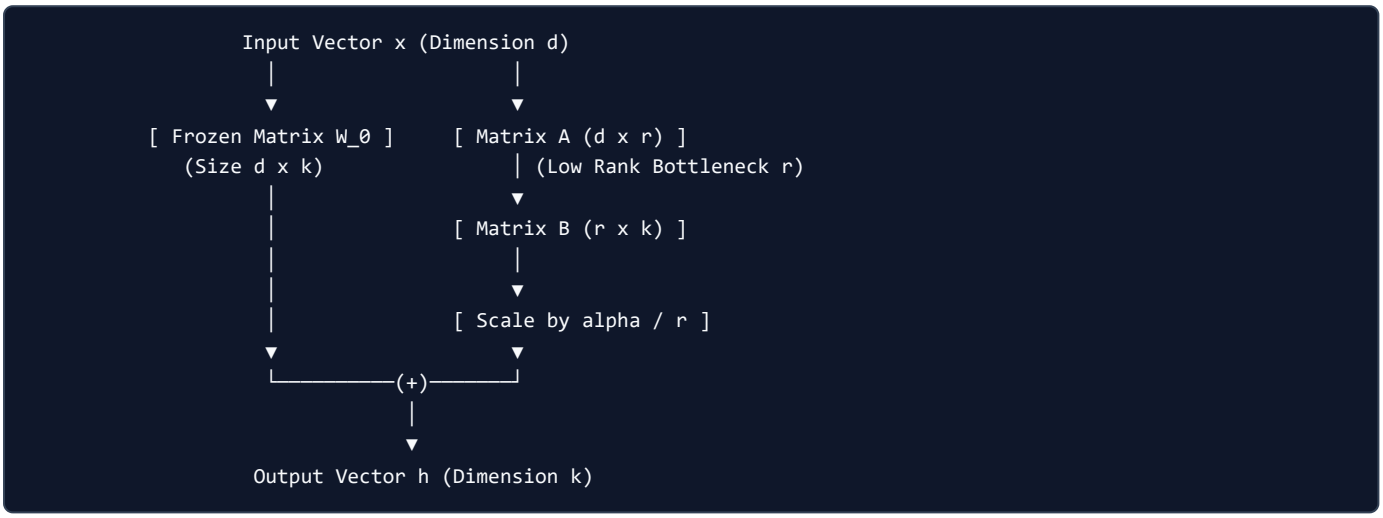
7.2 Low-Rank Adaptation (LoRA) Derivation

Developed by Hu et al. (2021), LoRA hypothesizes that weight updates during domain adaptation have a low "intrinsic dimension". For a pre-trained weight matrix $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$, LoRA decomposes the weight update $\Delta\mathbf{W}$ into two low-rank matrices:

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}(\mathbf{B} \cdot \mathbf{A})$$

Where:

- $\mathbf{W}_0 \in \mathbb{R}^{d \times k}$ is **frozen** and receives no gradients.
- $\mathbf{B} \in \mathbb{R}^{d \times r}$ is initialized to all zeros.
- $\mathbf{A} \in \mathbb{R}^{r \times k}$ is initialized with Gaussian random values.
- $r \ll \min(d, k)$ is the rank (typically $r \in \{8, 16\}$).
- α is a constant scaling hyperparameter.



7.3 Memory Reduction with LoRA

Consider a projection matrix where $d = 4096, k = 4096$:

- Full Parameters: $4096 \times 4096 = 16,777,216$ parameters.
- LoRA Parameters ($r = 8$): $(4096 \times 8) + (8 \times 4096) = 65,536$ parameters.
- **Reduction**: Trainable parameters are reduced by $256\times$ **(over 99.6% savings!)**.

MODULE 8: Multi-Node Slurm Orchestration for Distributed LLM Training & Serving

8.1 Multi-Node Slurm Batch Script for LoRA Fine-Tuning

```
#!/bin/bash
#SBATCH --job-name=llm_lora_finetune
#SBATCH --partition=ai_cluster
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=32
#SBATCH --time=04:00:00
#SBATCH --output=logs/llm_tune_%j.log
#SBATCH --error=logs/llm_tune_%j.err

# Load Intel oneAPI Environment
source /opt/intel/oneapi/setvars.sh --force > /dev/null 2>&1

# Distributed Environment Variables
export MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
export MASTER_PORT=29500
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

# Launch distributed fine-tuning with torchrun
torchrun \
    --nnodes=$SLURM_JOB_NUM_NODES \
    --nproc_per_node=$SLURM_NTASKS_PER_NODE \
    --rdzv_id=$SLURM_JOB_ID \
    --rdzv_backend=c10d \
    --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT \
    finetune_lora_openvino.py \
    --model_id "Qwen/Qwen2.5-0.5B-Instruct" \
    --dataset "medical_qa_dataset.json" \
    --lora_rank 8 \
    --batch_size 4 \
    --epochs 3
```

8.2 Production Serving with `ov::genai::LLMPipeline`

Once fine-tuned and quantized, models are deployed as low-latency microservices using OpenVINO GenAI:

```
import openvino_genai as ov_genai

# Load quantized model directly into CPU/AMX execution plugin
pipe = ov_genai.LLMPipeline("./qwen_openvino_int4", "CPU")

# Configure generation parameters
config = ov_genai.GenerationConfig()
config.max_new_tokens = 256
config.temperature = 0.7
config.top_p = 0.9

# Stream tokens directly to client
def streamer(subword):
    print(subword, end="", flush=True)
    return False # Continue generation

print("Prompt: What is high performance computing?")
pipe.generate("What is high performance computing?", config, streamer)
print("\n")
```

🔬 SELF-ASSESSMENT QUESTIONS

1. Why is single-token auto-regressive decoding fundamentally memory-bandwidth bound rather than compute bound?
2. How does Intel AMX achieve an $8\times$ throughput increase over AVX-512 during the prefill phase?
3. What architectural advantages does Habana Gaudi's on-die 24x 100/200GbE networking provide over traditional PCIe-based GPU clusters?
4. In Weight-Only Quantization (WOQ), explain why quantizing weights to INT4 yields near-maximum speedup even though activations remain in FP16.
5. Derive the parameter savings of applying LoRA with rank $r = 16$ on a 4096×4096 linear weight matrix.

GLA University HPC Masterclass | Day 5 Study Guide Completed